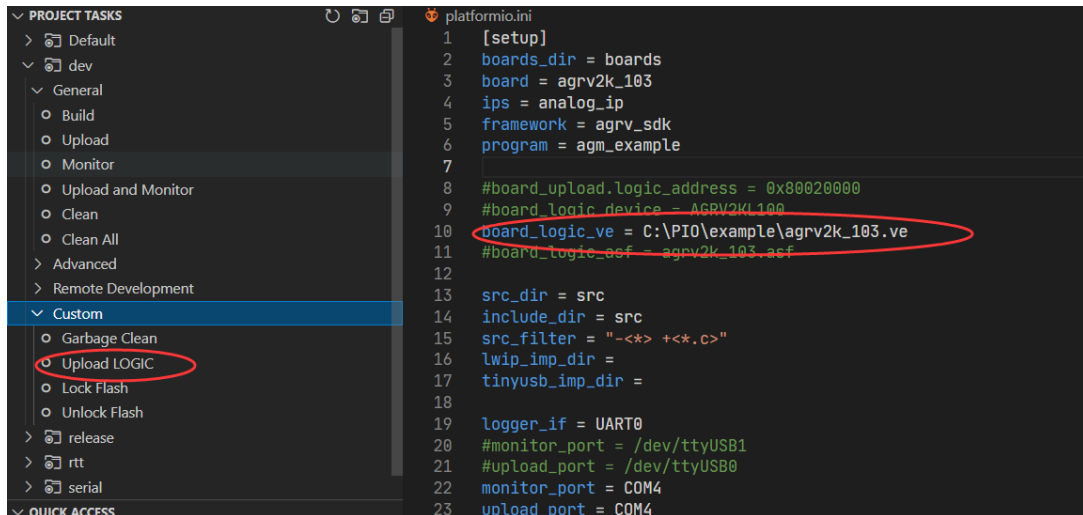


AGRV2K 逻辑设置

1. 引脚设置



通过逻辑改变引脚定义：在platformio.ini中指定： board_logic_ve，然后点击PROJECT TASKS下的Upload LOGIC。VE文件格式为每个引脚一行，每行由Function pin和Device pin组成，可参考 AgRV_pio\platforms\AgRV\boards\agrv2k_103.ve。Device pin 由封装决定，一般为PIN_xx的形式。

Function pin 列表：

SYSCCLK	System clock frequency (MHz)	Required
HSECLK	HSE oscillator frequency (MHz)	Default:8
BUSCLK	AHB bus clock frequency (MHz)	Optional, must be divided from SYSCCLK
USB0	USB mode, must be host, device, or OTG	Optional
USB0_ID	USB external ID pin	USB mode is OTG if this pin is defined
EXT_INT0	External Interrupt 0	
EXT_INT1		
EXT_INT2		
EXT_INT3		
EXT_INT4		
EXT_INT5		
EXT_INT6		
EXT_INT7	External Interrupt 7	
MAC0_CLK_OUT	MAC output clock pin	Can be used to drive MAC PHY

GPIO0_0	SPI0_SI_IO0(INOUT)	
GPIO0_1	SPI0_SO_IO1(INOUT)	
GPIO0_2	SPI0_WPN_IO2(INOUT)	
GPIO0_3	SPI0_HOLDN_IO3(INOUT)	
GPIO0_4	SPI1_SI_IO0(INOUT)	
GPIO0_5	SPI1_SO_IO1(INOUT)	
GPIO0_6	SPI1_WPN_IO2(INOUT)	
GPIO0_7	SPI1_HOLDN_IO3(INOUT)	
GPIO1_0	GPTIMER0_CH0(INOUT)	
GPIO1_1	GPTIMER0_CH1(INOUT)	
GPIO1_2	GPTIMER0_CH2(INOUT)	
GPIO1_3	GPTIMER0_CH3(INOUT)	
GPIO1_4	GPTIMER1_CH0(INOUT)	
GPIO1_5	GPTIMER1_CH1(INOUT)	
GPIO1_6	GPTIMER1_CH2(INOUT)	
GPIO1_7	GPTIMER1_CH3(INOUT)	
GPIO2_0	GPTIMER2_CH0(INOUT)	
GPIO2_1	GPTIMER2_CH1(INOUT)	
GPIO2_2	GPTIMER2_CH2(INOUT)	
GPIO2_3	GPTIMER2_CH3(INOUT)	
GPIO2_4	GPTIMER3_CH0(INOUT)	
GPIO2_5	GPTIMER3_CH1(INOUT)	
GPIO2_6	GPTIMER3_CH2(INOUT)	
GPIO2_7	GPTIMER3_CH3(INOUT)	
GPIO3_0	GPTIMER4_CH0(INOUT)	
GPIO3_1	GPTIMER4_CH1(INOUT)	
GPIO3_2	GPTIMER4_CH2(INOUT)	
GPIO3_3	GPTIMER4_CH3(INOUT)	
GPIO3_4	I2C0_SCL(INOUT)	
GPIO3_5	I2C0_SDA(INOUT)	
GPIO3_6	I2C1_SCL(INOUT)	
GPIO3_7	I2C1_SDA(INOUT)	
GPIO4_0	MAC0_MDIO(INOUT)	
GPIO4_1	GPTIMER0_BRK(INPUT)	
GPIO4_2	GPTIMER0_ETR(INPUT)	
GPIO4_3	GPTIMER0_OCREF_CLR(INPUT)	
GPIO4_4	GPTIMER1_BRK(INPUT)	
GPIO4_5	GPTIMER1_ETR(INPUT)	SPI0_SCK(OUTPUT)

GPIO4_6	GPTIMER1_OCREF_CLR(INPUT)	SPI0_CSN(OUTPUT)
GPIO4_7	GPTIMER2_BRK(INPUT)	SPI1_SCK(OUTPUT)
GPIO5_0	GPTIMER2_ETR(INPUT)	SPI1_CSN(OUTPUT)
GPIO5_1	GPTIMER2_OCREF_CLR(INPUT)	GPTIMER0_CHN0(OUTPUT)
GPIO5_2	GPTIMER3_BRK(INPUT)	GPTIMER0_CHN1(OUTPUT)
GPIO5_3	GPTIMER3_ETR(INPUT)	GPTIMER0_CHN2(OUTPUT)
GPIO5_4	GPTIMER3_OCREF_CLR(INPUT)	GPTIMER0_CHN3(OUTPUT)
GPIO5_5	GPTIMER4_BRK(INPUT)	GPTIMER1_CHN0(OUTPUT)
GPIO5_6	GPTIMER4_ETR(INPUT)	GPTIMER1_CHN1(OUTPUT)
GPIO5_7	GPTIMER4_OCREF_CLR(INPUT)	GPTIMER1_CHN2(OUTPUT)
GPIO6_0	UART0_NUARTCTS(INPUT)	GPTIMER1_CHN3(OUTPUT)
GPIO6_1	UART0_UARTRXD(INPUT)	GPTIMER2_CHN0(OUTPUT)
GPIO6_2	UART1_NUARTCTS(INPUT)	GPTIMER2_CHN1(OUTPUT)
GPIO6_3	UART1_UARTRXD(INPUT)	GPTIMER2_CHN2(OUTPUT)
GPIO6_4	UART2_NUARTCTS(INPUT)	GPTIMER2_CHN3(OUTPUT)
GPIO6_5	UART2_UARTRXD(INPUT)	GPTIMER3_CHN0(OUTPUT)
GPIO6_6	UART3_NUARTCTS(INPUT)	GPTIMER3_CHN1(OUTPUT)
GPIO6_7	UART3_UARTRXD(INPUT)	GPTIMER3_CHN2(OUTPUT)
GPIO7_0	UART4_NUARTCTS(INPUT)	GPTIMER3_CHN3(OUTPUT)
GPIO7_1	UART4_UARTRXD(INPUT)	GPTIMER4_CHN0(OUTPUT)
GPIO7_2	CAN0_INT_IN(INPUT)	GPTIMER4_CHN1(OUTPUT)
GPIO7_3	CAN0_RX0(INPUT)	GPTIMER4_CHN2(OUTPUT)
GPIO7_4	MAC0_PHY_INTB(INPUT)	GPTIMER4_CHN3(OUTPUT)
GPIO7_5	MAC0_TX_CLK(INPUT)	UART0_NUARTRTS(OUTPUT)
GPIO7_6	MAC0_RX_CLK(INPUT)	UART0_UARTTXD(OUTPUT)
GPIO7_7	MAC0_RXD0(INPUT)	UART1_NUARTRTS(OUTPUT)
GPIO8_0	MAC0_RXD1(INPUT)	UART1_UARTTXD(OUTPUT)
GPIO8_1	MAC0_RXD2(INPUT)	UART2_NUARTRTS(OUTPUT)
GPIO8_2	MAC0_RXD3(INPUT)	UART2_UARTTXD(OUTPUT)
GPIO8_3	MAC0_RX_DV(INPUT)	UART3_NUARTRTS(OUTPUT)
GPIO8_4	MAC0_RX_ER(INPUT)	UART3_UARTTXD(OUTPUT)
GPIO8_5	MAC0_CRS(INPUT)	UART4_NUARTRTS(OUTPUT)
GPIO8_6	MAC0_COL(INPUT)	UART4_UARTTXD(OUTPUT)
GPIO8_7	CAN0_TX0(OUTPUT)	
GPIO9_0		
GPIO9_1	MAC0_TXD0(OUTPUT)	
GPIO9_2	MAC0_TXD1(OUTPUT)	
GPIO9_3	MAC0_TXD2(OUTPUT)	

GPIO9_4	MAC0_TXD3(OUTPUT)	
GPIO9_5	MAC0_TX_EN(OUTPUT)	
GPIO9_6	MAC0_TX_ER(OUTPUT)	
GPIO9_7	MAC0_MDC(OUTPUT)	

每一行的GPIO和外设引脚同一时间只能使用一个，可以在程序中使用 GPIO_AF_ENABLE(MAC0_MDC) 或 GPIO_AF_DISABLE(MAC0_MDC) 来打开或关闭外设。同一行的输入和输出外设引脚可以同时使用，例如UART1_UARTRXD可以和GPTIMER2_CHN2同时使用，但使用了二者任意一个均不可使用GPIO6_3。

2. 预定义IP

```
platformio.ini
1  [setup]
2  boards_dir = boards
3  board = agrv2k_103
4  ips = analog_ip
5  framework = agrv_sdk
6  program = agm_example
7
8  #board_upload.logic_address = 0x80020000
9  #board_logic_device = AGRV2KL100
10 board_logic_ve = C:\PIO\example\agrv2k_103.ve
11 #board_logic_asf = agrv2k_103.asf
12 ip_name = analog_ip
13
```

在platformio.ini中加入：ip_name，然后同上点击Upload LOGIC。

3. 自定义IP

```
PLATFORMIO  ...  C example.c  platformio.ini X
PROJECT TASKS
  > Default
  > dev
    > General
      > Build
      > Upload
      > Monitor
      > Upload and Monitor
      > Clean
      > Clean All
    > Advanced
    > Remote Development
    > Custom
      > Garbage Clean
      > Upload LOGIC
      > Prepare LOGIC
      > Lock Flash
      > Unlock Flash
  > release

platformio.ini
1  [setup]
2  boards_dir = boards
3  board = agrv2k_103
4  ips = analog_ip
5  framework = agrv_sdk
6  program = agm_example
7
8  #board_upload.logic_address = 0x80020000
9  #board_logic_device = AGRV2KL100
10 board_logic_ve = C:\PIO\example\agrv2k_103.ve
11 #board_logic_asf = agrv2k_103.asf
12 logic_dir = logic
13 ip_name = custom_ip
14
15 src_dir = src
16 include_dir = src
17 src_filter = "-<*> +<*.c>"
18 lwip_imp_dir =
19 tinyusb_imp_dir =
20
```

在platformio.ini中加入logic_dir和ip_name。logic_dir是存放自定义逻辑的子目录，如果该目录不存在将自动创建。ip_name为自定义逻辑的模块名称。然后点击PROJECT TASKS下的 Prepare LOGIC，工具将在指定的目录下产生一个Supra工程，工程文件以上图为例是 C:\PIO\example\logic\agrv2k_103.proj。Supra软件可以通过AgRV_pio目录下的快捷方式 Supra.exe启动，在软件中打开这个工程文件并选择 Tools -> Compile，详细使用请参见Supra的使用手册。在Supra工具成功编译完成之后，再点击Upload LOGIC更新逻辑配置。以上图为例，逻辑配置文件将产生于C:\PIO\example\logic\agrv2k_103.bin。

自定义逻辑的ve文件中除上述 "Function_pin Device_pin" 的引脚设置外，还可以加入另外两种引脚设置：

- 1. 与自定义逻辑连接的顶层管脚，格式为：“Name Device_pin:DIR”，DIR可以是INPUT，OUTPUT，或者INOUT。工具将把一个指定方向的管脚加入自定义逻辑，并连接到对应的芯片顶层管脚。
- 2. 与自定义逻辑连接的某个MCU功能，格式为：“Function_pin Name”。根据Funtion pin的方向，工具将自动在Name后面加入后缀 _in (用于驱动Function pin输入)，以及 _out_data/_out_en (用于接收Function pin的输出数据和使能) 作为自定义逻辑的管脚。

ve文件配置	自定义逻辑模块信号	
Name Device_pin:INPUT	input Name	
Name Device_pin:OUTPUT	output Name	
Name Device_pin:INOUT	inout Name	
Function_pin Name	output Name_in	当Function pin为INPUT
	input Name_out_data	当Function pin为OUTPUT
	input Name_out_en	
	output Name_in	当Function pin为INOUT
	input Name_out_data	
	input Name_out_en	

自动创建的Supra工程缺省将包含两个Verilog文件，分别对应芯片顶层和自定义逻辑模块。以上面为例名称分别是agrv2k_103.v和custom_ip.v。其中芯片顶层文件agrv2k_103.v每次运行Prepare LOGIC的时候都会根据ve文件自动生成覆盖，请勿手工修改。自定义逻辑模块文件custom_ip.v则不会被覆盖，如果已经存在，新生成的文件将被命名为custom_ip_tmpl.v。生成的文件中包含根据ve文件生成的所有自定义逻辑模块管脚。

例如一个下面的ve配置：

```
1 txd_top PIN_67:OUTPUT
2 rxd_top PIN_66:INPUT
3 UART4_UARTTXD txd_ip
4 UART4_UARTRXD rxd_ip
```

将生成如下的自动逻辑模块：

```
1 module custom_ip (
2     output      txd_top,
3     input       rxd_top,
4     output      rxd_ip_in,
5     input       txd_ip_out_data,
6     input       txd_ip_out_en,
7     ....
```

除根据ve文件生成的管脚外，自定义模块中还包含以下信号与MCU连接：

信号	方向	
sys_clock	input	MCU system clock
bus_clock	input	Optional clock defined in ve. Same as sys_clock if undefined.
resetn	input	System reset, active low
stop	input	MCU has entered stop mode, active high
mem_ahb_*	MCU as master	AHB bus controlled by MCU. Address range: 0x60000000 – 0x7FFFFFFF
slave_ahb_*	MCU as slave	AHB bus controlled by logic. Can access all valid address.
ext_dma_*	DMAC as controller	4 external DMA request signals, defined as EXT_DMA0_REQ – EXT_DMA3_REQ in SDK. Detailed waveform can be found in DMAC document.
local_int[3:0]	output	4 local interrupt signals, defined as LOCAL_INT0_IRQn – LOCAL_INT3_IRQn in SDK. Can be enabled with SDK function INT_EnableIntLocal().